

Linear regression:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
df = pd.read_csv("csvfile.csv")
df.head()
X = df.drop(columns=["target"])
y = df["math_score"]
num_features = X.select_dtypes(exclude="object").columns
cat_features = X.select_dtypes(include="object").columns
```

```
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
```

```
numerical_transformer = StandardScaler()
hot_transformer = OneHotEncoder()
```

```
preprocessor = ColumnTransformer([
    ("OneHotEncoder", hot_transformer, cat_features),
    ("StandardScaler", numerical_transformer, num_features)
])
X = preprocessor.fit_transform(X)
X.shape
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size=0.2, random_state=42)
def evaluate_model(actual, predicted):
    mae = mean_absolute_error(actual, predicted)
    rmse = mean_squared_error(actual, predicted)
    r2_squared = r2_score(actual, predicted)
    return mae, rmse, r2_squared
model = LinearRegression(fit_intercept=True)
model.fit(X_train, Y_train)
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
```

```
train_mae, train_rmse, train_r2 = evaluate_model(Y_train, y_train_pred)
test_mae, test_rmse, test_r2 = evaluate_model(Y_test, y_test_pred)
```

```
print("Linear Regression")
print(f"Training: mae: {train_mae} rmse: {train_rmse} r2: {train_r2}")
print(f"Testing: mae: {test_mae} rmse: {test_rmse} r2: {test_r2}")
score = r2_score(Y_test, y_test_pred)*100
print(f"Accuracy r2 score: {score}")
```

```
plt.scatter(Y_test, y_test_pred)
plt.xlabel("Actual")
plt.ylabel("Predicted")
```

```
sns.regplot(x=Y_test, y=y_test_pred, ci=None, color="red")
pred_df = pd.DataFrame({"Actual Value":Y_test, "Predicted Value":y_test_pred, "Difference":Y_test-
y_test_pred})
pred_df
```

```
new_student = pd.DataFrame({
"gender": ["female"],
"race_ethnicity": ["group C"],
"parental_level_of_education": ["associate's degree"],
"lunch": ["standard"],
"test_preparation_course": ["none"],
"reading_score": [66],
"writing_score": [67],
"math_score": [59]
})
```

```
X_new = new_student.drop(columns=["target"])
X_new_processed = preprocessor.transform(X_new)
```

```
predicted_math_score = model.predict(X_new_processed)[0]
predicted_math_score
```

```
actual_math_score = new_student["math_score"].values[0]
difference = actual_math_score - predicted_math_score
```

```
comparison = pd.DataFrame({
"Actual Math Score": [actual_math_score],
"Predicted Math Score": [predicted_math_score],
"Difference": [difference]
})
```

```
comparison
```